



## Objetivos de Aprendizagem

Ao final deste módulo, o aluno será capaz de:

- Compreender os conceitos fundamentais de sistemas distribuídos de banco de dados
- Identificar diferentes arquiteturas de distribuição de dados
- Entender estratégias de fragmentação e replicação
- Aplicar conceitos de transações distribuídas
- Reconhecer problemas de consistência e suas soluções
- Analisar trade-offs entre disponibilidade, consistência e tolerância a partições (Teorema CAP)
- Implementar soluções práticas usando conceitos de sistemas distribuídos



## Índice

- [1. Introdução aos Sistemas Distribuídos](#)
- [2. Arquiteturas de Banco de Dados Distribuídos](#)
- [3. Distribuição de Dados](#)
- [4. Transações Distribuídas](#)
- [5. Teorema CAP e Propriedades BASE](#)
- [6. Algoritmos de Consenso](#)
- [7. Replicação de Dados](#)
- [8. Exemplos Práticos do Dia a Dia](#)
- [9. Problemas Comuns e Soluções](#)
- [10. Referências Bibliográficas](#)

---

## 1. Introdução aos Sistemas Distribuídos

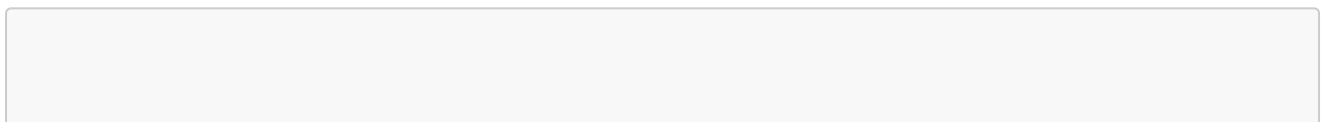
### 1.1 O que é um Sistema de Banco de Dados Distribuído?

Um **Sistema de Banco de Dados Distribuído (SBDD)** é uma coleção de múltiplos bancos de dados logicamente inter-relacionados, distribuídos em uma rede de computadores.

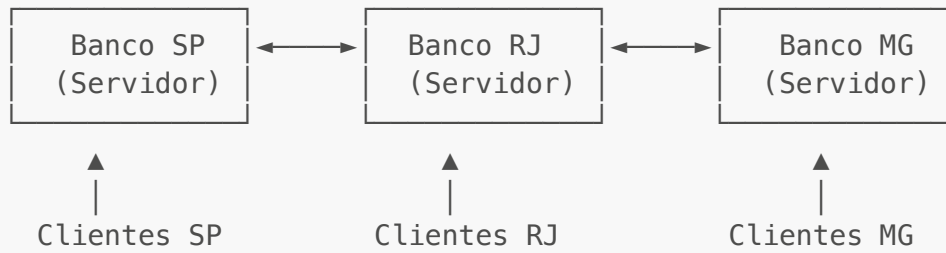
#### Características Principais:

- **Distribuição Física:** Os dados estão armazenados em múltiplos locais físicos
- **Transparência:** O usuário não precisa saber onde os dados estão armazenados
- **Autonomia:** Cada nó pode operar independentemente
- **Integração:** Os dados podem ser acessados e combinados de diferentes locais

#### Exemplo Acadêmico:



### Sistema Bancário Nacional:



#### Exemplo do Dia a Dia:

**Netflix:** Quando você assiste um filme na Netflix, o conteúdo não vem de um único servidor:

- Seus dados de perfil podem estar em um servidor nos EUA
- O catálogo de filmes está replicado em servidores CDN pelo mundo
- O histórico de visualização pode estar em outro datacenter
- O filme em si é carregado do servidor mais próximo de você

**O que pode acontecer:** Se o servidor de perfis cair, você ainda pode assistir filmes (usando cache local), mas não pode alterar suas preferências até o servidor voltar.

## 1.2 Por que Usar Sistemas Distribuídos?

### Vantagens:

1. **Desempenho:** Processamento paralelo e proximidade com usuários
2. **Disponibilidade:** Se um nó falha, outros continuam operando
3. **Escalabilidade:** Fácil adicionar novos nós conforme demanda cresce
4. **Tolerância a Falhas:** Redundância protege contra perda de dados

### Desvantagens:

1. **Complexidade:** Gerenciamento mais difícil que sistemas centralizados
2. **Custo:** Infraestrutura e manutenção mais caras
3. **Consistência:** Difícil manter todos os nós sincronizados
4. **Segurança:** Mais pontos de ataque possíveis

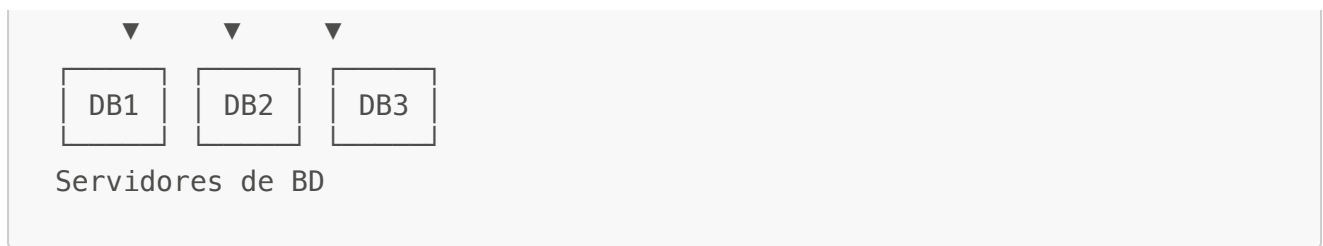
---

## 2. Arquiteturas de Banco de Dados Distribuídos

### 2.1 Arquitetura Cliente-Servidor

A arquitetura mais comum, onde clientes fazem requisições a servidores.





### Exemplo Acadêmico:

Sistema universitário onde:

- **Servidor 1:** Dados acadêmicos (notas, disciplinas)
- **Servidor 2:** Dados financeiros (mensalidades, bolsas)
- **Servidor 3:** Dados de biblioteca (acervo, empréstimos)

### Exemplo do Dia a Dia:

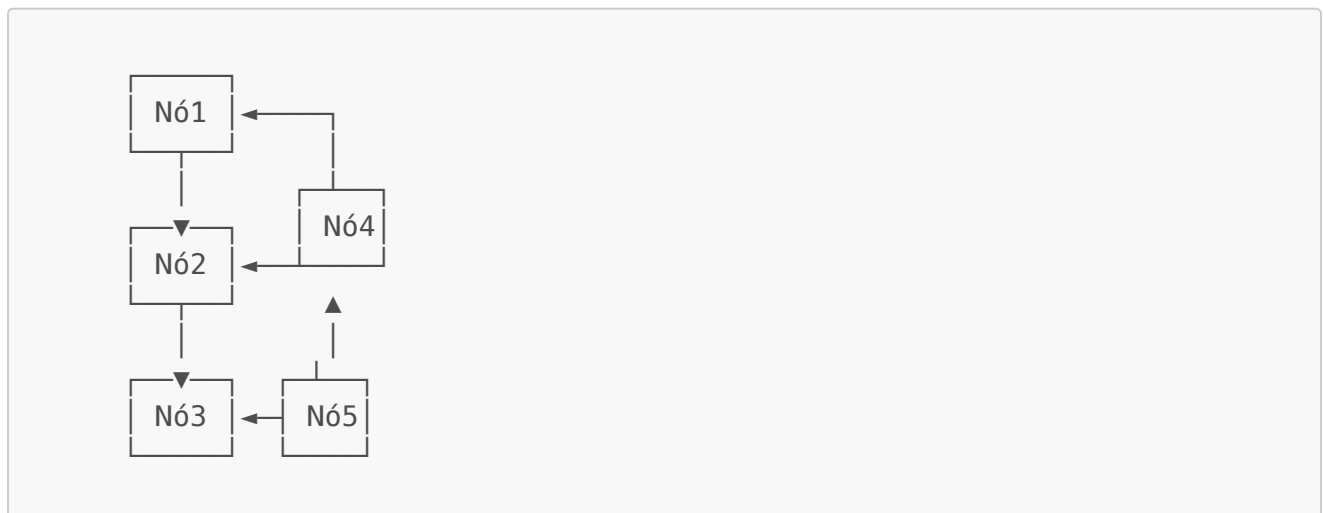
#### Instagram:

- Servidor 1: Armazena suas fotos
- Servidor 2: Armazena comentários e curtidas
- Servidor 3: Armazena dados de perfil e seguidores

**O que pode acontecer:** Você posta uma foto, mas ela demora a aparecer para seus seguidores porque está sendo replicada entre servidores.

## 2.2 Arquitetura Peer-to-Peer (P2P)

Todos os nós têm papel igual, sem servidor central.



### Exemplo Acadêmico:

Blockchain e criptomoedas como Bitcoin, onde cada nó mantém uma cópia completa do ledger.

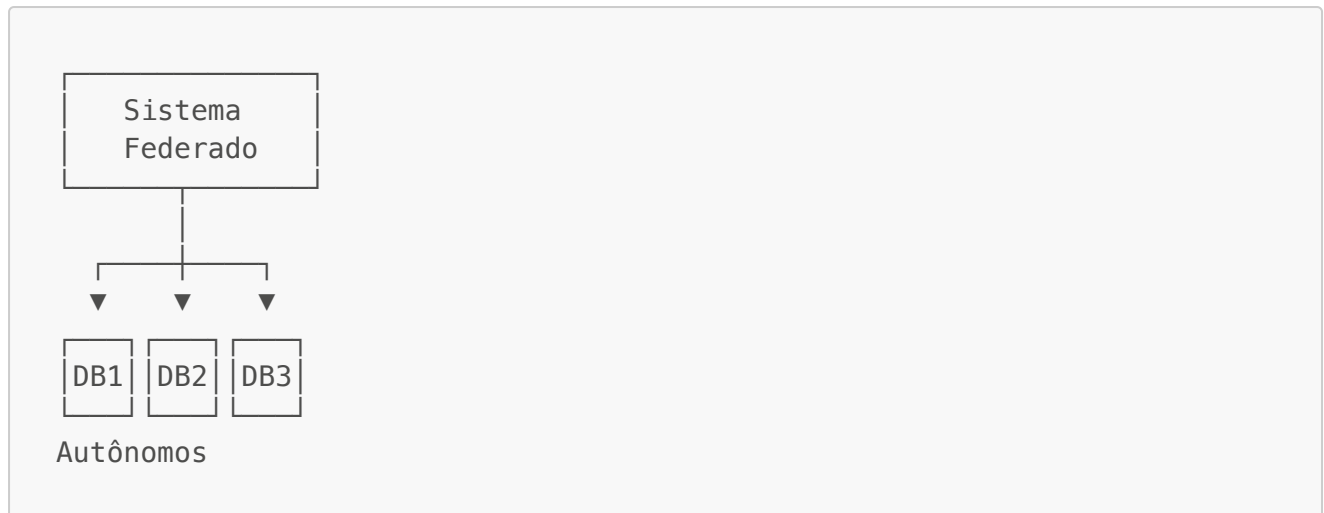
### Exemplo do Dia a Dia:

**BitTorrent:** Quando você baixa um arquivo grande, pedaços vêm de diferentes pessoas conectadas à rede, não de um servidor central.

**O que pode acontecer:** Se muitos nós saírem da rede, o download fica lento porque há menos fontes compartilhando os dados.

## 2.3 Arquitetura Federada

Bancos de dados autônomos cooperam, mas mantêm independência.



### Exemplo Acadêmico:

Sistema de saúde nacional onde hospitais mantêm seus próprios bancos, mas compartilham informações através de um sistema federado.

### Exemplo do Dia a Dia:

**Sistemas de Reserva de Passagens Aéreas:** Cada companhia tem seu próprio banco de dados, mas sites agregadores (Decolar, Kayak) consultam todos através de uma camada federada.

**O que pode acontecer:** Você busca um voo e vê disponibilidade, mas ao tentar comprar recebe "indisponível" porque o banco da companhia aérea foi atualizado após sua consulta.

---

## 3. Distribuição de Dados

### 3.1 Fragmentação Horizontal

Dividir tabelas em linhas (registros) distribuídas em diferentes nós.

### Exemplo Acadêmico:

```
-- Fragmentação por região geográfica
-- Nó São Paulo
CREATE TABLE clientes_sp AS
SELECT * FROM clientes WHERE estado = 'SP';
```

```
-- Nó Rio de Janeiro
CREATE TABLE clientes_rj AS
SELECT * FROM clientes WHERE estado = 'RJ';

-- Nó Minas Gerais
CREATE TABLE clientes_mg AS
SELECT * FROM clientes WHERE estado = 'MG';
```

#### Exemplo do Dia a Dia:

##### Uber:

- Servidor América do Sul: Corridas no Brasil, Argentina, Chile
- Servidor América do Norte: Corridas nos EUA, Canadá, México
- Servidor Europa: Corridas em países europeus

**O que pode acontecer:** Se você está no Brasil e tenta ver o histórico de uma corrida que fez nos EUA, a consulta precisa ir até o servidor americano, podendo demorar mais.

**Solução Acadêmica:** Usar **índices globais** e **cache de consultas frequentes** para acelerar acesso a dados remotos.

### 3.2 Fragmentação Vertical

Dividir tabelas em colunas distribuídas em diferentes nós.

#### Exemplo Acadêmico:

```
-- Nó 1: Dados públicos
CREATE TABLE usuarios_publico AS
SELECT id, nome, email, data_cadastro
FROM usuarios;

-- Nó 2: Dados sensíveis (servidor seguro)
CREATE TABLE usuarios_privado AS
SELECT id, cpf, senha_hash, cartao_credito
FROM usuarios;
```

#### Exemplo do Dia a Dia:

##### E-commerce (Magazine Luiza, Mercado Livre):

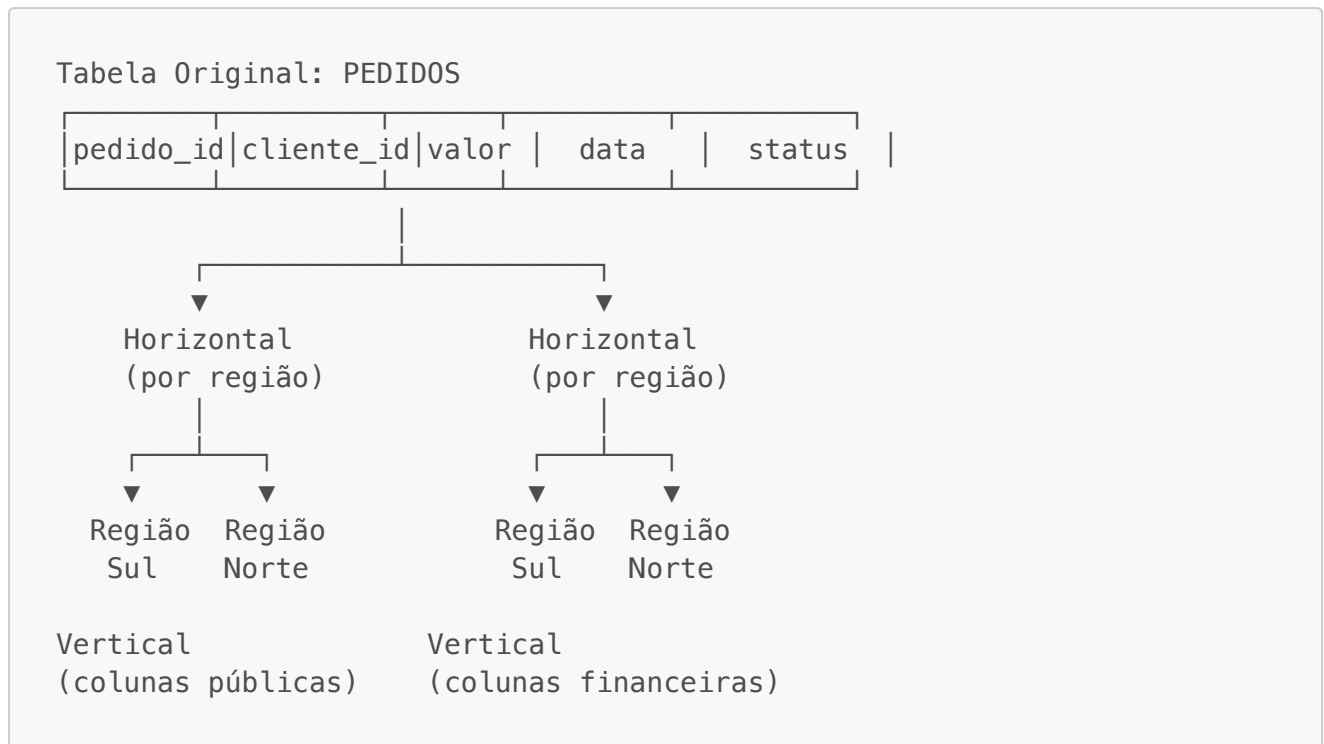
- Servidor público: Nome do produto, descrição, fotos, preço
- Servidor seguro: Dados de pagamento, cartão de crédito
- Servidor logístico: Estoque, localização em armazém

**O que pode acontecer:** Você vê um produto disponível, adiciona ao carrinho, mas na hora do pagamento o sistema diz que não há estoque. Isso ocorre porque o servidor de estoque ainda não sincronizou com o servidor público.

**Solução Acadêmica:** Implementar **verificação de estoque em tempo real** antes de mostrar disponibilidade, ou usar **locks distribuídos** durante a compra.

### 3.3 Fragmentação Mista (Híbrida)

Combina fragmentação horizontal e vertical.



## 4. Transações Distribuídas

### 4.1 Propriedades ACID em Ambiente Distribuído

Em sistemas distribuídos, manter ACID é desafiador:

- **Atomicidade:** Todas as operações em todos os nós devem ser completadas ou nenhuma
- **Consistência:** Todos os nós devem chegar ao mesmo estado final
- **Isolamento:** Transações concorrentes não devem interferir umas nas outras
- **Durabilidade:** Uma vez confirmada, a transação persiste em todos os nós

**Exemplo Acadêmico:**

```
-- Transferência bancária entre bancos diferentes
BEGIN TRANSACTION;

-- Nó Banco A (São Paulo)
UPDATE contas
```

```

SET saldo = saldo - 1000
WHERE conta_id = '12345-SP';

-- Nó Banco B (Rio de Janeiro)
UPDATE contas
SET saldo = saldo + 1000
WHERE conta_id = '67890-RJ';

COMMIT;

```

### Exemplo do Dia a Dia:

#### PIX (Sistema de Pagamento Instantâneo):

1. Você envia R\$ 100 do Banco Itaú para uma conta do Banco Bradesco
2. **Cenário problema sem controle adequado:**
  - Itaú debita R\$ 100 da sua conta (sucesso)
  - Rede cai antes de creditar no Bradesco
  - Dinheiro "desaparece" do sistema

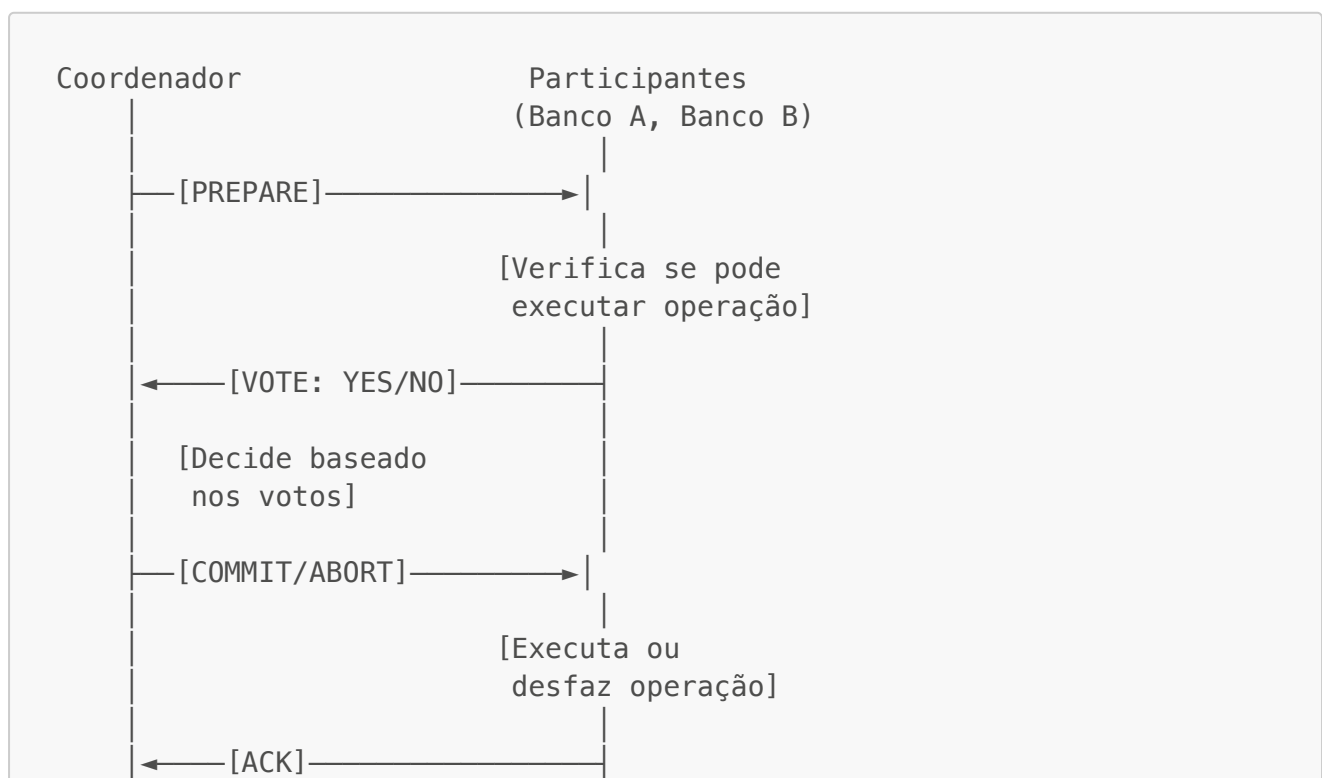
**O que aconteceu:** Violação da **Atomicidade** - uma parte da transação executou, outra não.

**Solução Acadêmica:** Usar **protocolo 2PC (Two-Phase Commit)** onde:

1. **Fase 1 (Prepare):** Pergunta a todos os bancos se podem executar
2. **Fase 2 (Commit/Abort):** Só executa se TODOS confirmarem

### 4.2 Protocolo 2PC (Two-Phase Commit)

O protocolo mais usado para garantir atomicidade distribuída.



## Exemplo do Dia a Dia Detalhado:

### Compra com Cartão de Crédito Online:

#### Fase 1 - PREPARE:

- Sistema verifica: Cartão tem limite? ✓
- Sistema verifica: Produto em estoque? ✓
- Sistema verifica: Endereço de entrega válido? ✓
- Sistema verifica: Gateway de pagamento disponível? ✓

#### Fase 2 - COMMIT:

- Deduz do limite do cartão
- Reserva produto no estoque
- Gera pedido de entrega
- Confirma pagamento

#### O que pode acontecer:

- **Problema:** O gateway de pagamento responde "VOTE: NO" na Fase 1
- **Solução:** ABORT - nada é executado, você recebe mensagem de erro
- **Resultado:** Integridade mantida, mas compra não realizada

## 4.3 Protocolo 3PC (Three-Phase Commit)

Melhoria do 2PC para evitar bloqueios em caso de falha do coordenador.

Fase 1: CAN-COMMIT (preparar para preparar)  
Fase 2: PRE-COMMIT (preparar de verdade)  
Fase 3: DO-COMMIT (executar)

## Exemplo do Dia a Dia:

### Reserva de Viagem Completa (Voo + Hotel + Carro):

#### Fase 1 - CAN-COMMIT:

- Consulta disponibilidade em todos os sistemas
- "Voo disponível? Posso reservar?"
- "Hotel disponível? Posso reservar?"
- "Carro disponível? Posso reservar?"

#### Fase 2 - PRE-COMMIT:

- Reserva temporária por 10 minutos
- Bloqueia voo, hotel e carro



- Aguarda confirmação de pagamento

### Fase 3 - DO-COMMIT:

- Pagamento aprovado
- Confirma todas as reservas
- Envia e-mails de confirmação

### O que pode acontecer:

- **Problema:** Você clica em "Confirmar" mas a rede cai na Fase 2
- **Solução do 3PC:** Reservas temporárias expiram em 10 minutos automaticamente
- **Resultado:** Nada é cobrado, assentos voltam a ficar disponíveis

**Vantagem sobre 2PC:** No 2PC, se o coordenador cair na Fase 2, participantes ficam bloqueados sem saber se devem commitar ou abortar. No 3PC, há um timeout.

## 5. Teorema CAP e Propriedades BASE

### 5.1 Teorema CAP (Brewer's Theorem)

**Teorema:** Em um sistema distribuído, é impossível garantir simultaneamente:

- **Consistency** (Consistência)
- **Availability** (Disponibilidade)
- **Partition Tolerance** (Tolerância a Partições)

Você pode ter no máximo 2 dos 3.



### Classificação de Sistemas:

#### CP (Consistency + Partition Tolerance):

- Prioriza consistência sobre disponibilidade
- Se há partição de rede, recusa requisições

- Exemplo: Sistemas bancários tradicionais

#### **AP (Availability + Partition Tolerance):**

- Prioriza disponibilidade sobre consistência
- Aceita inconsistências temporárias
- Exemplo: Redes sociais, DNS

#### **CA (Consistency + Availability):**

- Só funciona sem partições de rede
- Praticamente impossível em sistemas distribuídos reais
- Exemplo teórico: Banco de dados centralizado

#### **Exemplo Acadêmico:**

Sistema de Votação Online:

Opção CP (Consistência + Partição):

- Garante que cada voto é contado uma única vez
- Se houver problema de rede, sistema fica indisponível
- Escolha: Melhor não votar do que votar duas vezes

Opção AP (Disponibilidade + Partição):

- Sistema sempre disponível para votar
- Pode aceitar votos duplicados temporariamente
- Reconcilia depois quando rede voltar
- Escolha: Aceitável para enquetes informais, não para eleições oficiais

#### **Exemplos do Dia a Dia:**

##### **Sistema Bancário (CP):**

- **Cenário:** Você tenta transferir R\$ 500, mas há problema de rede entre servidores
- **O que acontece:** Transação é recusada, você vê mensagem "Serviço temporariamente indisponível"
- **Por quê:** Banco PREFERE ficar indisponível do que arriscar inconsistência (dinheiro duplicado)
- **Solução Acadêmica:** Usar **algoritmos de consenso distribuído** (Paxos, Raft) para garantir que todos os nós concordem antes de processar

##### **Instagram/Facebook (AP):**

- **Cenário:** Você posta uma foto, mas há partição de rede entre datacenters
- **O que acontece:** Foto é postada, mas demora a aparecer para amigos em outras regiões
- **Por quê:** Prioriza disponibilidade, permite **consistência eventual**
- **Resultado:** Após alguns segundos/minutos, todos veem a mesma coisa
- **Solução Acadêmica:** Usar **timestamps vetoriais** e **resolução de conflitos** para sincronizar

### WhatsApp (AP com eventual consistência):

- **Cenário:** Você e seu amigo postam mensagens simultaneamente em um grupo com perda de conexão
- **O que acontece:** Ambos veem suas próprias mensagens primeiro, depois sincroniza
- **Por quê:** Melhor experiência do usuário (sempre funciona)
- **Trade-off:** Ordem das mensagens pode variar entre usuários por alguns segundos

## 5.2 Propriedades BASE

Uma alternativa ao ACID para sistemas distribuídos que priorizam disponibilidade:

- **Basically Available:** Sistema sempre responde (mesmo que com dados desatualizados)
- **Soft state:** Estado do sistema pode mudar sem input (devido a eventual consistency)
- **Eventually consistent:** Sistema convergirá para consistência dado tempo suficiente

### Exemplo Acadêmico:

E-commerce com BASE:

T0: Produto tem 10 unidades em estoque  
T1: Cliente A compra 5 unidades (Servidor São Paulo)  
T2: Cliente B compra 5 unidades (Servidor Rio de Janeiro)  
T3: Cliente C tenta comprar 3 unidades

Com ACID (CP): Cliente C recebe "Sem estoque" imediatamente

Com BASE (AP): Cliente C consegue comprar, estoque fica negativo temporariamente

T4: Sistema detecta problema e cancela pedido do Cliente C  
T5: Consistência eventual alcançada

### Exemplo do Dia a Dia:

#### Amazon - Carrinho de Compras:

1. **T0:** Produto tem 2 unidades restantes
2. **T1:** Você adiciona ao carrinho em São Paulo
3. **T2:** Outra pessoa adiciona ao carrinho no Rio (antes da replicação)
4. **T3:** Ambos têm o produto no carrinho (aparentemente 4 unidades vendidas!)
5. **T4:** Você clica em "Finalizar compra" primeiro → Sucesso
6. **T5:** Outra pessoa clica depois → "Produto indisponível no estoque"

#### O que aconteceu:

- Sistema usou **BASE** para dar disponibilidade
- Ambos puderam navegar e adicionar ao carrinho
- Verificação real de estoque só na finalização (**eventual consistency**)

### Solução Acadêmica:

- Implementar **reserva otimista**: Avisa "apenas 2 unidades restantes, finalize rápido"
- Usar **locks pessimistas** no checkout final
- Implementar **fila de espera** para produtos muito disputados

---

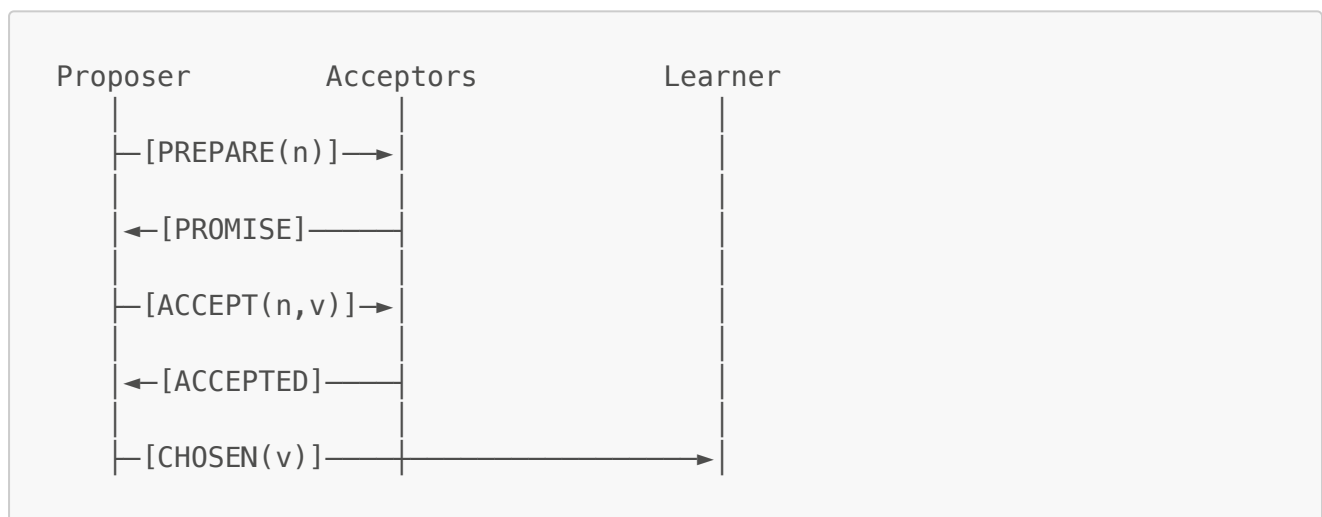
## 6. Algoritmos de Consenso

### 6.1 Paxos

Algoritmo clássico para alcançar consenso em ambiente distribuído com falhas.

#### Papéis:

- **Proposer**: Propõe valores
- **Acceptor**: Aceita ou rejeita propostas
- **Learner**: Aprende o valor escolhido



#### Exemplo do Dia a Dia:

**Google Chubby** (sistema de locks distribuídos usado internamente):

- Garante que apenas um servidor seja o "mestre" por vez
- Se mestre cai, Paxos eleger novo mestre
- Todos os servidores concordam quem é o novo mestre

#### O que pode acontecer:

- Servidor Mestre em SP cai
- Paxos inicia eleição
- Servidores votam: RJ recebe 3 votos, MG recebe 2 votos
- RJ se torna novo mestre
- Todos os clientes são redirecionados para RJ

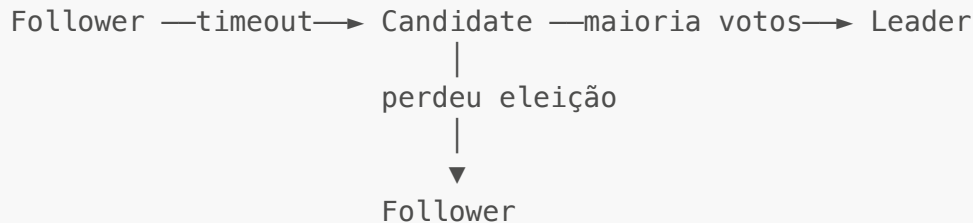
### 6.2 Raft

Algoritmo mais simples e compreensível que Paxos, com mesmo objetivo.

#### Estados:

- **Leader:** Líder único que recebe todas as escritas
- **Follower:** Replica dados do líder
- **Candidate:** Nó que está tentando se tornar líder

Eleição de Líder:



#### Exemplo do Dia a Dia:

**etcd** (usado por Kubernetes):

- Armazena configuração de clusters
- Usa Raft para garantir consenso
- Se o líder cai, novo líder é eleito em ~1 segundo

#### Cenário Real:

1. Cluster tem 5 nós: A (líder), B, C, D, E (followers)
2. Nó A cai (servidor com problema)
3. B detecta timeout (não recebe heartbeat do líder)
4. B vira candidato e pede votos
5. C, D, E votam em B (maioria: 3 de 5)
6. B se torna novo líder
7. Aplicações continuam funcionando com <1s de interrupção

#### Solução Acadêmica:

- Usar **número ímpar de nós** (3, 5, 7) para evitar empates
- **Heartbeat:** Líder envia mensagem a cada 100ms
- **Election timeout:** 150-300ms aleatório para evitar eleições simultâneas

### 6.3 Algoritmo de Consenso Bizantino (PBFT)

Para ambientes onde nós podem ser maliciosos (não apenas falhar).

#### Exemplo Acadêmico:

Blockchain e criptomoedas (Bitcoin, Ethereum).

## Exemplo do Dia a Dia:

### Bitcoin:

- Mineradores competem para adicionar bloco à cadeia
- **Problema:** E se um minerador for desonesto?
- **Solução:** Prova de Trabalho (Proof of Work)
  - Minerador precisa resolver problema computacional difícil
  - Outros validam solução (fácil de verificar)
  - Bloco só é aceito se maioria concordar

### O que pode acontecer:

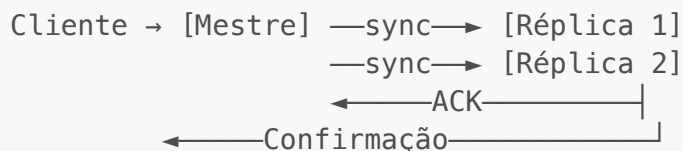
- Minerador malicioso tenta gastar mesma moeda duas vezes
- Cria transação falsa
- Rede rejeita porque não tem Prova de Trabalho válida
- Mesmo se criar uma, maioria da rede rejeitará

---

## 7. Replicação de Dados

### 7.1 Tipos de Replicação

#### Replicação Síncrona



**Vantagens:** Garantia de consistência forte

**Desvantagens:** Maior latência, menor disponibilidade

### Exemplo do Dia a Dia:

**Sistema Bancário Core:** Quando você transfere dinheiro, todas as réplicas são atualizadas antes de você receber confirmação. Se uma réplica está fora, transação falha.

#### Replicação Assíncrona



**Vantagens:** Baixa latência, alta disponibilidade

**Desvantagens:** Possível perda de dados, inconsistência temporária

### Exemplo do Dia a Dia:

**Twitter:** Você twitta, recebe confirmação imediata, mas pode demorar segundos para aparecer para todos os seguidores globalmente.

### O que pode acontecer:

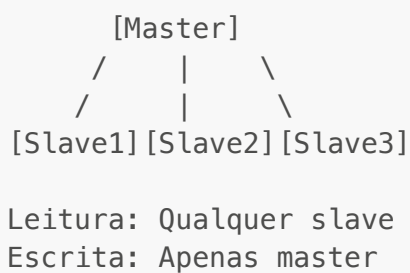
- Você posta tweet às 10:00:00
- Servidor primário confirma às 10:00:01
- Servidor primário cai às 10:00:02 (antes de replicar)
- Tweet é perdido (raro, mas possível)

### Solução Acadêmica:

- Usar **Write-Ahead Log (WAL)**: Escreve em log antes de replicar
- **Réplicas com lag monitorado**: Alerta se atraso > limite
- **Promote réplica atualizada**: Em caso de falha, promove réplica com menos lag

## 7.2 Estratégias de Replicação

### Master-Slave (Primary-Replica)



### Exemplo do Dia a Dia:

#### MySQL Replication:

- Aplicação web lê de slaves (rápido, distribui carga)
- Aplicação escreve no master (garante consistência)

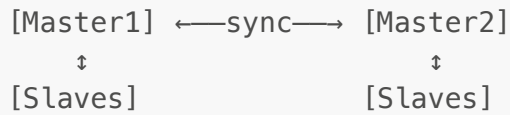
### Problema comum: Read-after-Write inconsistency

- Você posta comentário (escreve no master)
- Recarrega página (lê de slave ainda não sincronizado)
- Seu comentário não aparece por alguns segundos

### Solução Acadêmica: Session Affinity

- Durante X segundos após escrita, lê do master
- Ou marca timestamp e slave só responde se atualizado

### Multi-Master



### Exemplo do Dia a Dia:

#### Google Docs (Operational Transformation):

- Você e colega editam documento simultaneamente
- Cada um escreve em seu "master" local
- Sistema sincroniza e resolve conflitos

#### O que pode acontecer:

- Você escreve "Olá" na linha 1
- Colega escreve "Oi" na linha 1 (ao mesmo tempo)
- Sistema detecta conflito
- **Resolução:** Última escrita vence, ou mantém ambas marcadas como conflito

#### Solução Acadêmica: Operational Transformation (OT) ou CRDT (Conflict-free Replicated Data Types)

- Transforma operações concorrentes para convergir
- Garante que todos eventualmente vejam mesmo resultado

## 8. Exemplos Práticos do Dia a Dia

### 8.1 Streaming de Vídeo (YouTube, Netflix)

#### Arquitetura Distribuída:

```

Usuário → [CDN Edge Server Mais Próximo]
          ↓ (se não tem cache)
          [Regional Server]
          ↓ (se não tem)
          [Origin Server]

```

#### Estratégias:

- **Replicação Geográfica:** Vídeos populares em todos os servidores
- **Cache em Camadas:** Edge → Regional → Origin
- **Predição:** Pré-carrega vídeos populares antes de serem solicitados

#### Cenário do Dia a Dia:

1. Novo episódio de série popular lançado às 9h
2. Milhões tentam assistir simultaneamente



3. **Sem distribuição:** Servidor único colapsa

4. **Com distribuição:**

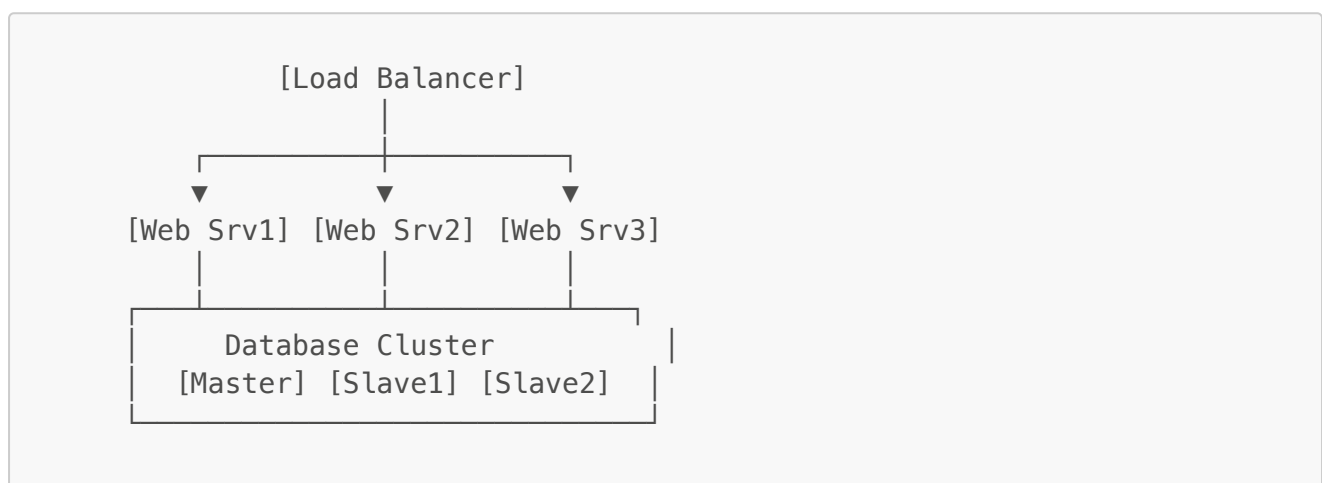
- Replicado em 1000+ servidores globalmente
- Cada usuário acessa servidor mais próximo
- Carga distribuída, streaming suave

**O que pode acontecer:**

- **Problema:** CDN em sua região fica offline
- **Solução automática:** Roteado para CDN de região próxima
- **Resultado:** Pequeno aumento em latência (50ms → 150ms), mas continua funcionando

## 8.2 E-commerce (Black Friday)

**Arquitetura durante picos:**



**Estratégias:**

- **Read Replicas:** Consultas de produtos em slaves
- **Cache Redis:** Produtos mais vistos em memória
- **Queue de Pedidos:** Pedidos vão para fila, processados assincronamente
- **Eventual Consistency:** Estoque pode ficar ligeiramente impreciso

**Cenário Black Friday:**

1. **09:00:** Tráfego normal - 1.000 usuários/segundo
2. **10:00:** Início Black Friday - 50.000 usuários/segundo
3. **Sem preparação:** Sistema colapsa
4. **Com preparação:**
  - Auto-scaling adiciona 50 servidores web
  - Cache serve 95% das consultas (produtos populares)
  - Write queue absorve pico de pedidos
  - Processamento assíncrono completa pedidos em minutos

**O que pode acontecer:**

- **Problema:** Você e outra pessoa clicam em "Comprar" no último produto (ao mesmo tempo)

- **Sem controle:** Ambos compram (overselling)
- **Com lock distribuído:**
  - Primeiro request adquire lock
  - Segundo request espera
  - Primeiro completa compra
  - Segundo recebe "Produto esgotado"

### Solução Acadêmica: Distributed Lock com Redis ou Database Row-Level Locking

```
-- Tentativa de compra com lock
BEGIN TRANSACTION;

SELECT estoque
FROM produtos
WHERE id = 123
FOR UPDATE; -- Lock pessimista

UPDATE produtos
SET estoque = estoque - 1
WHERE id = 123 AND estoque > 0;

-- Se UPDATE afetou 0 linhas, não havia estoque
COMMIT;
```

## 8.3 Redes Sociais (Facebook, Instagram)

### Desafios:

- Bilhões de usuários
- Postagens em tempo real
- Notificações instantâneas
- Diferentes regiões geográficas

### Arquitetura:



### Estratégias:

- **Geo-Distribution:** Dados do usuário replicados geograficamente
- **Timeline:** Pré-computada assincronamente (não em tempo real)

- **Lazy Replication:** Postagens replicadas sob demanda

#### Cenário do Dia a Dia:

##### Amigo nos EUA posta foto às 15:00 (horário de Brasília):

1. Foto salva em datacenter US (0,01s)
2. Notificação push enviada a seguidores online (0,1s)
3. Replicação para datacenter BR (2-5s)
4. Você no BR abre app e vê foto (quase instantâneo se já replicou)

#### O que pode acontecer:

- **Problema:** Seu amigo posta foto, mas você não vê imediatamente
- **Motivo:** Eventual consistency - ainda não replicou para seu datacenter
- **Solução:** Timeline é atualizada quando você abre o app e sistema verifica atualizações

#### Solução Acadêmica: Push vs Pull

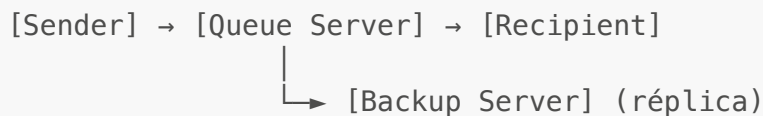
- **Push:** Servidor notifica cliente quando há atualização (WebSocket, Push Notification)
- **Pull:** Cliente pergunta periodicamente se há atualizações (Polling)
- **Híbrido:** Polling normal + Push para eventos importantes

### 8.4 Sistemas de Mensageria (WhatsApp, Telegram)

#### Requisitos:

- Entrega garantida de mensagens
- Ordem preservada
- Disponibilidade alta
- Criptografia ponta-a-ponta

#### Arquitetura:



#### Estratégias:

- **Message Queue:** Mensagens armazenadas até confirmação de entrega
- **Checkpoints:** Cliente envia ACK ao receber
- **Offline Queue:** Guarda mensagens enquanto destinatário offline
- **Multi-Device Sync:** Sincroniza entre celular, web, desktop

#### Cenário do Dia a Dia:

1. Você envia mensagem às 10:00
2. Destinatário está offline
3. Mensagem fica em queue do servidor

4. Destinatário fica online às 10:30
5. Servidor entrega mensagem
6. Destinatário envia ACK
7. Você vê "✓✓" (entregue e lido)

#### O que pode acontecer:

- **Problema:** Você envia mensagem, mas não recebe confirmação
- **Motivo:** Servidor está replicando, ou destinatário sem internet estável
- **Solução:** Mensagem fica com ícone "🕒" (enviando)
- **Retry:** Cliente tenta reenviar automaticamente até sucesso

#### Solução Acadêmica: At-Least-Once Delivery

- Garante entrega, mas pode duplicar
- Cliente detecta duplicatas via ID único
- Melhor duplicar que perder mensagem

```
Idempotência:  
msg_id: 12345  
Servidor recebe 12345 → Processa  
Servidor recebe 12345 → Ignora (duplicata)
```

---

## 9. Problemas Comuns e Soluções

### 9.1 Split-Brain (Cérebro Dividido)

**Problema:** Rede particiona em dois grupos, cada um acha que é o primário.

```
Antes da partição:  
[Master]—[Slave1]  
|  
[Slave2]  
  
Depois da partição de rede:  
[Master]      [Slave1]  
              |  
              [Slave2]  
  
Slave2 se promove a Master!  
Agora há 2 Masters! (Conflito)
```

#### Exemplo do Dia a Dia:

Sistema bancário com 2 datacenters (SP e RJ) perde conexão entre eles:

- SP acha que RJ caiu, promove seu backup a mestre

- RJ acha que SP caiu, promove seu backup a mestre
- Ambos processam transações diferentes
- Quando rede volta, há conflitos (mesma conta modificada diferentemente)

## Soluções Acadêmicas:

### 1. Quorum

Total: 5 nós  
 Quorum: 3 (maioria)

Partição A: 2 nós → NÃO pode operar ( $< 3$ )  
 Partição B: 3 nós → PODE operar ( $\geq 3$ )

### 2. Fencing (STONITH - Shoot The Other Node In The Head)

Master detecta que pode estar em partição menor  
 → Desliga a si mesmo para evitar split-brain

### 3. Witness Node (Árbitro)

[Master]  $\longleftrightarrow$  [Witness]  $\longleftrightarrow$  [Slave]

Partição: Master perde conexão com Slave  
 Master pergunta a Witness: "Ainda estou mestre?"  
 Witness: "Não vejo Slave, você é mestre"

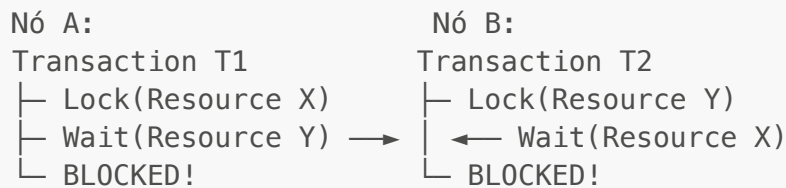
## Implementação Prática:

```
# Pseudo-código de verificação de quorum
def can_operate(nodes_alive, total_nodes):
    quorum = (total_nodes // 2) + 1
    if nodes_alive >= quorum:
        return True
    else:
        # Entre em modo read-only
        return False

# Exemplo:
# 5 nós, 3 alive → OK (3 > 2.5)
# 5 nós, 2 alive → FAIL (2 < 3)
```

## 9.2 Deadlock Distribuído

**Problema:** Transações esperam umas pelas outras em diferentes nós.



**Exemplo do Dia a Dia:**

**Reserva de passagem + hotel:**

- Usuário A: Bloqueia quarto 101 (RJ), tenta bloquear voo 2030 (SP)
- Usuário B: Bloqueia voo 2030 (SP), tenta bloquear quarto 101 (RJ)
- **Deadlock:** Ambos esperando eternamente

**Soluções Acadêmicas:**

### 1. Timeout

Se transação espera mais de X segundos → ABORT  
Vantagem: Simples  
Desvantagem: Escolha arbitrária do timeout

### 2. Detecção de Ciclos (Wait-For Graph)

T1 → T2 → T3 → T1 (Ciclo detectado!)  
↓  
Aborta transação mais nova (T3)

### 3. Ordenação de Recursos (Prevention)

Regra: Sempre adquirir locks em ordem alfabética

Correto:

T1: Lock(Hotel) → Lock(Voo)

T2: Lock(Hotel) → Lock(Voo)

Se T1 pegou Hotel primeiro, T2 espera  
Quando T1 libera, T2 adquire  
SEM DEADLOCK!

## Implementação Prática:

```
-- Solução com timeout no PostgreSQL
BEGIN;
SET lock_timeout = '5s';

LOCK TABLE reservas_hotel IN SHARE ROW EXCLUSIVE MODE;
LOCK TABLE reservas_voo IN SHARE ROW EXCLUSIVE MODE;

-- Se demorar mais de 5s, aborta automaticamente
COMMIT;
```

### 9.3 Perda de Dados por Falha Simultânea

**Problema:** Mestre e réplicas sincronizadas caem antes de replicar.

```
T0: [Master] escreve X = 10
T1: [Master] confirma ao cliente (X = 10 salvo!)
T2: [Master] replicando para [Slave]...
T3: [Master] cai antes de replicar
T4: [Slave] promovido a Master, mas tem X = 5 (valor antigo)

DADO PERDIDO: Cliente acha que salvou X = 10, mas está X = 5
```

#### Exemplo do Dia a Dia:

Você faz depósito bancário de R\$ 1000:

1. Sistema confirma: "Depósito realizado!"
2. Servidor primário cai antes de replicar
3. Servidor backup assume
4. Seu saldo não mostra os R\$ 1000

#### Soluções Acadêmicas:

##### 1. Replicação Síncrona (Espera confirmação de pelo menos N réplicas)

```
Master escreve X = 10
Master aguarda ACK de pelo menos 2 slaves
Slave1: ACK ✓
Slave2: ACK ✓
→ Agora confirma ao cliente
```

**Trade-off:** Maior latência, mas garantia mais forte

##### 2. Write-Ahead Log (WAL) em disco durável

1. Escrever em log em disco (fsync)
2. Confirmar ao cliente
3. Aplicar no banco de dados
4. Replicar

Se master cai, replay do WAL recupera dados

### 3. Distributed Log (como Apache Kafka)

Cliente → [Log Distribuído] → [Databases]  
(Kafka)

Log é replicado em múltiplos nós ANTES de confirmar  
Databases leem do log e aplicam

#### Implementação Prática:

```
# Pseudo-código de escrita com garantia
def write_with_guarantee(data, min_replicas=2):
    # 1. Escrever no WAL local (disco)
    wal.write(data)
    wal.fsync() # Força escrita física em disco

    # 2. Replicar para slaves
    acks = []
    for slave in slaves:
        acks.append(slave.replicate(data))

    # 3. Esperar confirmação de pelo menos min_replicas
    if len([ack for ack in acks if ack.success]) >= min_replicas:
        return "Success"
    else:
        return "Failure - not enough replicas"
```

### 9.4 Inconsistência de Leitura (Read Skew)

**Problema:** Ler dados de diferentes nós em momentos diferentes gera visão inconsistente.

Conta A = R\$ 1000, Conta B = R\$ 500 (Total = R\$ 1500)

T0: Transação transfere R\$ 500 de A para B

T1: Cliente lê Conta A de N1 → R\$ 500 (já atualizado)

T2: Cliente lê Conta B de N2 → R\$ 500 (ainda não replicou)

Total que cliente vê = R\$ 1000 (mas deveria ser R\$ 1500!)



R\$ 500 "desapareceram" da perspectiva do cliente!

### Exemplo do Dia a Dia:

Seu salário é R\$ 5000, você transfere R\$ 2000 para poupança:

- App mostra conta corrente: R\$ 3000 ✓ (atualizada)
- App mostra poupança: R\$ 1000 × (deveria ser R\$ 3000)
- Você pensa que perdeu R\$ 2000!

### Soluções Acadêmicas:

#### 1. Snapshot Isolation

Cliente inicia transação com timestamp T1  
Todas as leituras veem dados de T1 (snapshot consistente)  
Mesmo que outros modifiquem depois

```
-- PostgreSQL
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT * FROM conta WHERE id = 'A'; -- Vê snapshot T1
SELECT * FROM conta WHERE id = 'B'; -- Vê mesmo snapshot T1
COMMIT;
```

#### 2. Read from Master (Session Consistency)

Depois de escrever, sempre lê do mesmo nó (ou master)  
Garante que verá suas próprias escritas

#### 3. Versioning com Timestamps

Cada registro tem timestamp de modificação  
Cliente especifica: "Quero dados de no máximo T1"  
Sistema retorna apenas dados atualizados até T1

### Implementação Prática:

```
# Read-after-write consistency
class Session:
    def __init__(self):
```

```

self.last_write_timestamp = None
self.preferred_replica = None

def write(self, data):
    # Escreve no master
    master.write(data)
    self.last_write_timestamp = master.get_timestamp()
    self.preferred_replica = master

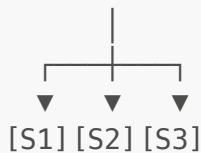
def read(self, key):
    if self.last_write_timestamp:
        # Lê de réplica que está atualizada
        for replica in replicas:
            if replica.is_synced_until(self.last_write_timestamp):
                return replica.read(key)
        # Fallback: lê do master
        return master.read(key)
    else:
        # Pode ler de qualquer réplica
        return random.choice(replicas).read(key)

```

## 9.5 Cascading Failures (Falhas em Cascata)

**Problema:** Falha de um componente causa sobrecarga que derruba outros.

[Load Balancer]



S1 cai → Carga vai para S2 e S3  
 S2 e S3 sobrecarregam → S2 cai  
 S3 sozinho não aguenta → S3 cai  
 SISTEMA TODO FORA!

**Exemplo do Dia a Dia:**

**Black Friday da Amazon (caso hipotético de problema):**

1. Servidor de pagamento em SP fica sobrecarregado
2. Requests são redirecionados para servidor do RJ
3. RJ também sobrecarrega e fica lento
4. Clientes fazem refresh (pensam que não funcionou)
5. Mais requests → MG também sobrecarrega
6. Efeito dominó: Sistema todo cai

**Soluções Acadêmicas:**

## 1. Circuit Breaker

Estado CLOSED (normal):  
Requests passam normalmente  
↓  
Muitas falhas detectadas  
↓  
Estado OPEN (aberto):  
Requests são rejeitados imediatamente  
"Service Unavailable" rápido  
↓  
Após timeout (ex: 30s)  
↓  
Estado HALF-OPEN (semi-aberto):  
Testa se serviço voltou  
↓  
Se sucesso → CLOSED  
Se falha → OPEN

```
class CircuitBreaker:
    def __init__(self, failure_threshold=5, timeout=30):
        self.failures = 0
        self.state = "CLOSED"
        self.last_failure_time = None

    def call(self, service_function):
        if self.state == "OPEN":
            if time.now() - self.last_failure_time > self.timeout:
                self.state = "HALF-OPEN"
            else:
                raise ServiceUnavailable("Circuit breaker open")

        try:
            result = service_function()
            self.failures = 0
            self.state = "CLOSED"
            return result
        except Exception:
            self.failures += 1
            self.last_failure_time = time.now()
            if self.failures >= self.failure_threshold:
                self.state = "OPEN"
            raise
```

## 2. Rate Limiting (Limitação de Taxa)

Máximo de 1000 requests/segundo por cliente

Se exceder → HTTP 429 Too Many Requests

Previne que um cliente malicioso ou bug derrube sistema

### 3. Bulkhead Pattern (Isolamento)

Recursos separados por tipo de operação:

Pool 1 (30 threads): Operações críticas (checkout)

Pool 2 (50 threads): Busca de produtos

Pool 3 (20 threads): Recomendações

Se busca sobrecarregar, checkout continua funcionando

### 4. Degradação Graciosa

Sistema detecta sobrecarga

→ Desabilita recursos não essenciais

Exemplo:

- Checkout: FUNCIONA ✓
- Recomendações: DESABILITADO ✗
- Reviews: DESABILITADO ✗
- Imagens HD: REDUZIDAS (apenas thumbnails)

Cliente consegue comprar, mas com experiência reduzida

### Implementação Prática:

```
# Rate limiting com Token Bucket
class RateLimiter:
    def __init__(self, rate=100, per=60): # 100 requests por 60
        segundos
        self.rate = rate
        self.per = per
        self.allowance = rate
        self.last_check = time.time()

    def allow_request(self):
        current = time.time()
        time_passed = current - self.last_check
        self.last_check = current
```

```

# Adiciona tokens baseado no tempo passado
self.allowance += time_passed * (self.rate / self.per)
if self.allowance > self.rate:
    self.allowance = self.rate

if self.allowance < 1.0:
    return False # Rate limit exceeded
else:
    self.allowance -= 1.0
    return True

# Uso:
limiter = RateLimiter(rate=1000, per=60)
if limiter.allow_request():
    process_request()
else:
    return "429 Too Many Requests"

```

## 10. O que são níveis de isolamento?

Níveis de isolamento definem **o quanto uma transação fica "isolada" das outras** enquanto está sendo executada.

Eles controlam:

- O que uma transação **pode ver**
- Quais **problemas de concorrência** podem acontecer

👉 Quanto maior o isolamento:

-  Mais segurança
-  Menos performance

## Problemas que os níveis tentam evitar

Antes de falar dos níveis, veja os problemas clássicos:

 Dirty Read (leitura suja)

Ler um dado **que ainda não foi confirmado (COMMIT)**.

 Non-Repeatable Read

Ler o **mesmo dado duas vezes** e obter **valores diferentes**.

 Phantom Read

Executar a **mesma consulta** e aparecerem **linhas novas ou sumidas**.

## Os 4 níveis de isolamento (SQL padrão)

### 1 READ UNCOMMITTED (menos seguro)

🚫 Permite **dirty read**

#### O que pode acontecer:

- Você lê dados que podem ser desfeitos (**ROLLBACK**)

#### Exemplo:

```
-- Transação A
BEGIN;
UPDATE contas SET saldo = 500 WHERE id = 1;
-- ainda não deu COMMIT

-- Transação B
SELECT saldo FROM contas WHERE id = 1;
-- pode ver 500 (mesmo sem COMMIT)
```

⚠ Pouco usado na prática.

---

### 2 READ COMMITTED (padrão na maioria dos bancos)

🚫 Só lê dados **confirmados**

❌ Não evita:

- Non-repeatable read
- Phantom read

#### Exemplo:

```
-- Transação A
BEGIN;
SELECT saldo FROM contas WHERE id = 1; -- 1000

-- Transação B
UPDATE contas SET saldo = 800 WHERE id = 1;
COMMIT;

-- Transação A
SELECT saldo FROM contas WHERE id = 1; -- 800 (mudou!)
```

✓ Evita dirty read

❌ O valor pode mudar durante a transação

---

---

### 3 REPEATABLE READ

🚫 Garante que **leituras de linhas já lidas não mudam**

✓ Evita:

- Dirty read
- Non-repeatable read

✗ Pode permitir:

- Phantom read (dependendo do banco)

**Exemplo:**

```
-- Transação A
BEGIN;
SELECT saldo FROM contas WHERE id = 1; -- 1000

-- Transação B
UPDATE contas SET saldo = 800 WHERE id = 1;
COMMIT;

-- Transação A
SELECT saldo FROM contas WHERE id = 1; -- ainda 1000
```

➡ A linha fica “congelada” para a transação A.

---

### 4 SERIALIZABLE (mais seguro)

🚫 Simula execução **uma transação por vez**

✓ Evita:

- Dirty read
- Non-repeatable read
- Phantom read

**Exemplo:**

```
-- Transação A
BEGIN;
SELECT * FROM contas WHERE saldo > 500;

-- Transação B
INSERT INTO contas (id, saldo) VALUES (3, 1000);
-- pode ser bloqueada ou falhar
```

🔒 Máximo isolamento

⚠️ Menor performance

## Tabela resumo

| Nível            | Dirty Read | Non-repeatable | Phantom |
|------------------|------------|----------------|---------|
| READ UNCOMMITTED | ✗          | ✗              | ✗       |
| READ COMMITTED   | ✓          | ✗              | ✗       |
| REPEATABLE READ  | ✓          | ✓              | ✗ *     |
| SERIALIZABLE     | ✓          | ✓              | ✓       |

(\*depende do banco)

## Como definir nível de isolamento

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Ou por transação:

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Qual usar?

- **READ COMMITTED** → aplicações comuns
- **REPEATABLE READ** → relatórios, cálculos
- **SERIALIZABLE** → operações críticas (financeiras)

Cenário base entre lock e MVCC

Tabela **contas**

```
id | saldo  
1  | 1000
```

Duas transações:

- **Transação A** → leitura



- Transação B → atualização

---

## 1 Lock tradicional (Lock-Based Concurrency)

---

Aqui o banco usa **bloqueios físicos** para controlar acesso.

---

### Exemplo: leitura com lock

Transação A

```
BEGIN;  
SELECT saldo FROM contas WHERE id = 1;
```

 O banco coloca um **lock de leitura** na linha.

---

Transação B

```
BEGIN;  
UPDATE contas SET saldo = 800 WHERE id = 1;
```

 **Bloqueada**, porque A está lendo.

 B só continua depois que A finalizar.

---

Transação A

```
COMMIT;
```

 Lock liberado

 Agora B pode atualizar

---

Características do lock tradicional

- Leitura **bloqueia escrita**
- Escrita **bloqueia leitura**
- Pode gerar:
  - Espera

- Lentidão
- Deadlocks

---

## 2 MVCC (Multi-Version Concurrency Control)

---

Usado por **PostgreSQL, Oracle, MySQL InnoDB**, etc.

👉 Em vez de bloquear leitura, o banco cria **versões dos dados**.

---

### Exemplo: leitura com MVCC

Transação A

```
BEGIN;  
SELECT saldo FROM contas WHERE id = 1;
```

- ➡ A vê o saldo **1000**
  - ➡ **Nenhum lock bloqueante** é aplicado
- 

Transação B (ao mesmo tempo)

```
BEGIN;  
UPDATE contas SET saldo = 800 WHERE id = 1;  
COMMIT;
```

➡ O banco cria **nova versão** da linha:

- Versão antiga: saldo = 1000
  - Versão nova: saldo = 800
- 

Transação A (continua)

```
SELECT saldo FROM contas WHERE id = 1;
```

- ➡ A **continua vendo 1000**
  - ➡ Mesmo após o COMMIT da B
- 

O que aconteceu?

- A está ligada a um **snapshot**
- B gravou uma nova versão
- **Leitura não bloqueou escrita**
- **Escrita não bloqueou leitura**

### 3 Comparação direta

| Aspecto                  | Lock Tradicional | MVCC     |
|--------------------------|------------------|----------|
| Leitura bloqueia escrita | ✓                | ✗        |
| Escrita bloqueia leitura | ✓                | ✗        |
| Performance em leitura   | ✗ pior           | ✓ melhor |
| Consistência             | ✓                | ✓        |
| Uso moderno              | ✗ raro           | ✓ padrão |

### 4 E quando o MVCC usa lock?

Mesmo com MVCC, **escrita ainda usa lock**.

Exemplo: duas escritas simultâneas

#### Transação A

```
UPDATE contas SET saldo = 900 WHERE id = 1;
```

#### Transação B

```
UPDATE contas SET saldo = 800 WHERE id = 1;
```

➡ Apenas **uma pode escrever por vez**

➡ A outra espera ou falha

🚫 MVCC **não elimina locks**, ele **elimina lock entre leitura e escrita**.

### 5 Phantom read: lock vs MVCC

Lock tradicional (REPEATABLE READ)

```
SELECT * FROM contas WHERE saldo > 500;
```

➡ Pode bloquear inserções novas

---

## MVCC

```
SELECT * FROM contas WHERE saldo > 500;
```

➡ Consulta usa snapshot

➡ Novas linhas não aparecem para a transação

---

## 6 Resumo mental

---

### Lock tradicional

- “Pare tudo enquanto eu acesso”
- Mais bloqueio
- Menos escalável

### MVCC

- “Cada um vê sua versão”
  - Pouco bloqueio
  - Alta concorrência
- 

## Regra prática

- Sistemas modernos → **MVCC**
- Sistemas legados ou específicos → **lock pesado**

## Resumo final

- Níveis de isolamento controlam **visibilidade de dados**
  - Mais isolamento = mais segurança
  - Menos isolamento = mais performance
  - Escolher o nível certo evita bugs difíceis
- 

## 11. Referências Bibliográficas

### Livros Acadêmicos

1. **Özsu, M. T., & Valduriez, P.** (2020). *Principles of Distributed Database Systems*. 4th Edition. Springer.
  - Referência definitiva sobre sistemas distribuídos de banco de dados
  - Cobre teoria e prática em profundidade
2. **Tanenbaum, A. S., & Van Steen, M.** (2017). *Distributed Systems: Principles and Paradigms*. 3rd Edition. CreateSpace.
  - Fundamentos de sistemas distribuídos
  - Algoritmos de consenso e sincronização
3. **Kleppmann, M.** (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
  - Abordagem prática e moderna
  - Exemplos de sistemas reais (Google, Amazon, Netflix)
  - Altamente recomendado para entender trade-offs
4. **Gray, J., & Reuter, A.** (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
  - Clássico sobre transações distribuídas
  - Fundamentos de ACID e recuperação
5. **Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G.** (2011). *Distributed Systems: Concepts and Design*. 5th Edition. Addison-Wesley.
  - Excelente para conceitos fundamentais
  - Exemplos didáticos

## Artigos Científicos Fundamentais

1. **Brewer, E.** (2000). "Towards Robust Distributed Systems" (PODC Keynote).
  - Introduz o Teorema CAP
  - Fundamento para entender trade-offs
2. **Lamport, L.** (1998). "The Part-Time Parliament". *ACM Transactions on Computer Systems*.
  - Artigo original do Paxos
  - Algoritmo fundamental de consenso
3. **Ongaro, D., & Ousterhout, J.** (2014). "In Search of an Understandable Consensus Algorithm" (USENIX ATC).
  - Introduz Raft
  - Alternativa mais compreensível ao Paxos
4. **Gilbert, S., & Lynch, N.** (2002). "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services". *ACM SIGACT News*.
  - Prova formal do Teorema CAP

5. **Vogels, W.** (2009). "Eventually Consistent". *Communications of the ACM*, 52(1).

- Conceitos de consistência eventual
- Perspectiva da Amazon (DynamoDB)

## Recursos Online

1. **Jepsen** (<https://jepsen.io/>)

- Análises de sistemas distribuídos reais
- Testes de consistência e partição

2. **Distributed Systems Course - MIT 6.824** (<https://pdos.csail.mit.edu/6.824/>)

- Curso completo com vídeos e labs
- Implementações práticas de algoritmos

3. **Amazon Builders' Library** (<https://aws.amazon.com/builders-library/>)

- Artigos sobre sistemas da Amazon
- Práticas reais de engenharia

4. **Google Research Papers** (<https://research.google/pubs/>)

- Papers sobre BigTable, Spanner, Chubby
- Sistemas distribuídos em escala global

## Sistemas Distribuídos em Produção (Para Estudo)

1. **Apache Cassandra** - Banco NoSQL distribuído (AP no CAP)
2. **Google Spanner** - Banco distribuído globalmente (CP no CAP)
3. **Amazon DynamoDB** - Banco NoSQL gerenciado (AP no CAP)
4. **CockroachDB** - SQL distribuído (CP no CAP)
5. **Apache Kafka** - Log distribuído
6. **etcd** - Armazenamento de configuração distribuída (usa Raft)
7. **Redis Cluster** - Cache distribuído

## Cursos e Tutoriais Recomendados

1. **Designing Data-Intensive Applications Book Club** - Discussões capítulo por capítulo
2. **Distributed Systems Lecture Series by Martin Kleppmann** - Vídeos gratuitos no YouTube
3. **Cloud Computing Specialization - Coursera** - Aplicações práticas de sistemas distribuídos



## Material Complementar

### Exemplos Práticos

Consulte o diretório [exemplos/](#) para scripts SQL e cenários práticos de:

- Configuração de replicação

- Fragmentação de dados
- Simulação de falhas
- Testes de consistência

## Exercícios

Consulte o diretório [exercicios/](#) para atividades práticas sobre:

- Projeto de arquiteturas distribuídas
- Análise de trade-offs CAP
- Implementação de protocolos de consenso
- Resolução de problemas comuns

---

## Contribuições

Este material é continuamente atualizado. Sugestões de melhorias são bem-vindas!

### Áreas para contribuição:

- Novos exemplos do dia a dia
- Casos de estudo de sistemas reais
- Exercícios práticos adicionais
- Correções e melhorias na documentação

---

## Sistemas Distribuídos: O Futuro dos Bancos de Dados

*De sistemas bancários a redes sociais, a distribuição é essencial para escalar na era moderna*

 Teoria sólida |  Exemplos reais |  Soluções práticas